SOLAR PANEL FAULT DETECTION AND ANALYSIS USING DEEP LEARNING

1. INTRODUCTION

The increasing global dependence on solar photovoltaic (PV) energy systems has made the reliable inspection and maintenance of solar panels a critical operational priority. Large-scale solar installations are continuously exposed to harsh environmental conditions — including ultraviolet radiation, thermal cycling, mechanical stress, dust accumulation, and partial shading — that progressively degrade panel performance and physical integrity. Faults such as surface cracking, hot-spot formation, and dust coverage can significantly reduce energy yield and, in severe cases, pose safety hazards. Detecting these faults early and accurately is therefore essential for cost-effective plant management.

Traditional inspection approaches, including manual visual surveys and periodic electrical performance testing through I–V curve measurements, are fundamentally limited in scalability for large installations comprising tens of thousands of panels. Thermal infrared imaging offers a superior non-invasive diagnostic modality, as different fault conditions manifest as distinct temperature anomalies visible in thermal photographs. The integration of deep learning with thermal imaging enables automated, accurate, and scalable fault detection, eliminating the need for manual intervention.

The present project implements a complete end-to-end deep learning pipeline for solar panel fault classification using a ResNet-18 model trained via transfer learning on a curated thermal image dataset. The system classifies panel images into four categories — Healthy, Cracked, Hot-Spot, and Dust-Covered — and provides confidence scores for each prediction. Grad-CAM is incorporated to generate spatial heatmaps explaining which image regions influenced the classification decision. Fault severity estimation, an alert system, model evaluation, and a Streamlit-based browser interface complete the pipeline.

2. SOFTWARE USED

The project is developed entirely in **Python 3**, the dominant language for scientific computing and machine learning. The deep learning framework used is **PyTorch**, which provides dynamic computational graphs and GPU-accelerated tensor operations, along with its companion library **TorchVision** for pre-trained model architectures and image transformation utilities.

OpenCV is used for image processing operations within the Grad-CAM module, including heatmap resizing, colour mapping, and overlay blending. **Matplotlib** handles all graphical output — input image display, heatmap rendering, and confusion matrix plotting. **scikit-learn** provides the classification_report and confusion_matrix utilities for model evaluation. The user interface is built using **Streamlit**, a Python-based framework for rapidly creating browser-accessible machine learning applications. All development is performed in **Visual Studio Code** with package management via pip.

3. THEORY / EXPLANATION

3.1 Convolutional Neural Networks and Transfer Learning

A Convolutional Neural Network (CNN) applies learned filter banks to input images to extract hierarchical spatial features — from low-level edges and textures in early layers to high-level semantic representations in deeper layers. Transfer learning initialises the network with weights pre-trained on ImageNet (over one million images, one thousand categories), then fine-tunes it on the target domain. This approach enables effective learning from a relatively small domain-specific dataset and dramatically reduces training time.

3.2 ResNet-18 Architecture

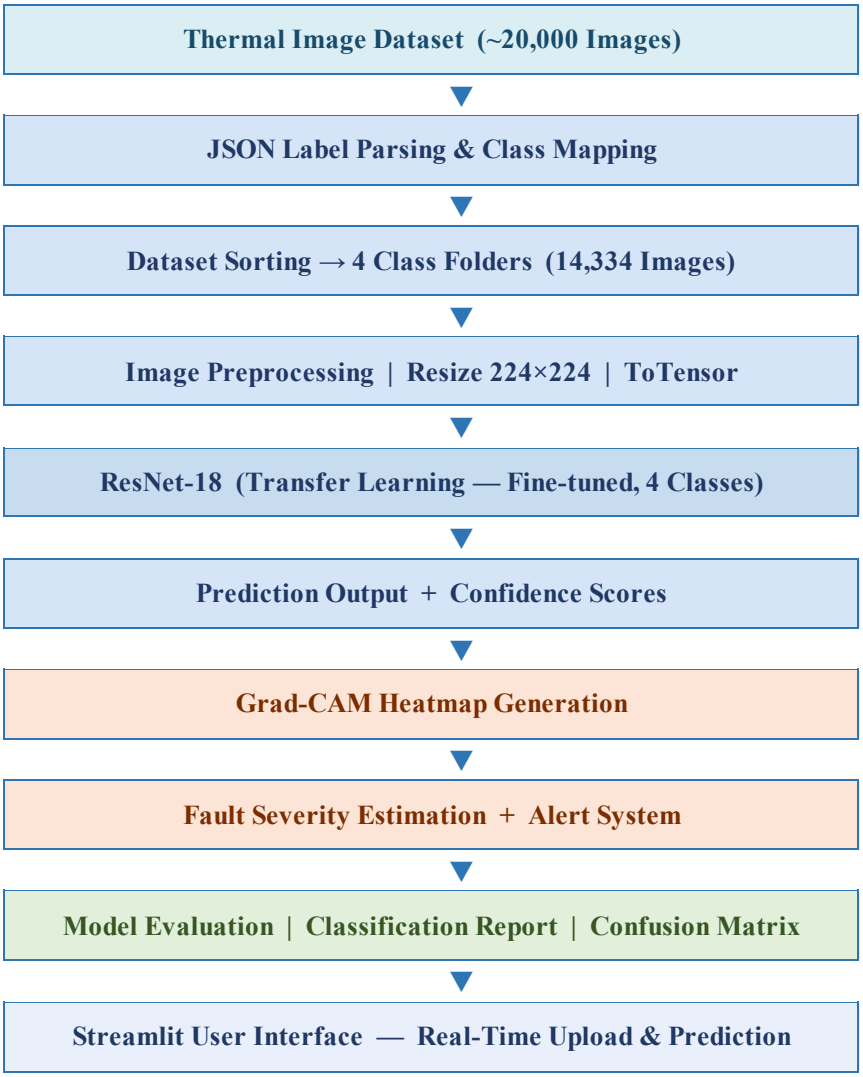
ResNet-18 is an 18-layer deep residual network that introduces shortcut (skip) connections between layers. These bypass connections allow gradients to flow directly during backpropagation, preventing the vanishing gradient problem that limits training of very deep networks. The network consists of four stages of residual blocks followed by global average pooling and a fully connected output layer. For this project, the output layer is replaced with a four-neuron linear layer corresponding to the four fault classes.

3.3 Grad-CAM — Explainability

Gradient-weighted Class Activation Mapping (Grad-CAM) computes the gradient of the predicted class score with respect to the final convolutional layer's activation maps. These gradients are globally average-pooled to produce per-channel importance weights. A ReLU-activated weighted combination of the feature maps yields a coarse spatial heatmap, upsampled to the input resolution. Warm colours (red, yellow) indicate regions that most strongly contributed to the prediction; cool colours (blue) indicate low influence.

3.4 Block Diagram

The system architecture follows a sequential processing pipeline from raw thermal image data to user-facing diagnostic output. The block diagram below illustrates all functional stages and the data flow between them:



The pipeline begins with approximately 20,000 raw thermal images accompanied by a JSON annotation file. A label parsing stage consolidates original fine-grained fault labels into the four target classes. After sorting, 14,334 valid images are preprocessed and fed into the ResNet-18 classifier. Prediction outputs and Grad-CAM heatmaps are generated in parallel, with the heatmap feeding the severity estimation and alert modules. A separate evaluation branch generates the classification report and confusion matrix. The Streamlit interface integrates all outputs for end-user access.

3.5 Fault Class Definitions

A **Healthy** panel exhibits uniform thermal emission with no anomalies. **Cracked** panels show localised temperature irregularities at fracture sites due to increased electrical resistance. **Hot-Spot** faults produce concentrated, intense overheating caused by cell mismatch or internal cell damage — the most critical fault type. The **Dust-Covered** class aggregates soiling, vegetation, and shadowing conditions, representing panels with partially obstructed irradiance and reduced thermal output.

4. PROGRAMME

The project is implemented as seven coordinated Python scripts executed in sequence. Each script encapsulates a distinct functional stage of the pipeline.

4.1 sort.py — Dataset Preparation

This script reads the JSON annotation file and applies a class consolidation mapping — 'No-Anomaly' → Healthy, 'Cracking' → Cracked, 'Hot-Spot*' → Hot-Spot, and 'Soiling/Vegetation/Shadowing' → Dust-Covered. Valid images are copied from the raw directory into class-specific subdirectories under data/train/. On completion, the script reports the total count of successfully copied images (14,334).

4.2 train.py — ResNet-18 Model Training

The training script loads the sorted dataset using PyTorch's ImageFolder utility, applies resize (224×224) and tensor conversion preprocessing, and trains a pre-trained ResNet-18 model with its output layer replaced by a four-class linear layer. The Adam optimiser at learning rate 0.001 and cross-entropy loss are used over five epochs with a batch size of 16. Training loss is printed per epoch, and the trained weights are saved as solar_model.pth.

4.3 predict.py — Single-Image Inference

This module loads the saved model in evaluation mode, preprocesses a query image, and performs a forward pass to produce four class probabilities via softmax. It prints confidence scores for all classes, announces the predicted class and confidence percentage, and issues a contextual alert: maintenance required for Cracked or Hot-Spot, cleaning recommended for Dust-Covered, and healthy status otherwise. Predictions below 60% confidence are flagged as uncertain.

4.4 gradcam.py — Grad-CAM Visualisation and Severity Estimation

Forward and backward hooks are registered on the final residual block's convolutional layer. After inference, gradients are average-pooled into importance weights, and a ReLU-

activated weighted feature map combination yields the heatmap. The heatmap is normalised, colourised using the JET palette, and blended with the original image at 40% opacity. The fraction of pixels exceeding a threshold of 0.6 determines severity — Low (<5%), Medium (5–15%), or High (>15%) — and an estimated power loss percentage is derived.

4.5 evaluate.py — Model Evaluation

The model is run over the full dataset with predictions and ground-truth labels collected. scikit-learn generates a per-class classification report (precision, recall, F1-score) and a confusion matrix rendered as a colour-coded grid with annotated cell counts, confirming approximately 85% overall accuracy.

4.6 train_mobilenet.py — Lightweight Model

A MobileNetV3-Small model is trained for one epoch with batch size 32 on the same dataset. Its depthwise separable convolutions and hard-swish activation reduce the parameter count substantially compared to ResNet-18, enabling edge deployment comparison. Weights are saved as mobilenet_model.pth.

4.7 app.py — Streamlit Interface

The Streamlit application renders a browser-based tool at localhost:8501. Users upload a thermal image, which is preprocessed and classified. The interface displays the predicted class, confidence score, per-class probability breakdown, and a colour-coded alert banner (red for maintenance-critical faults, orange for cleaning required, green for healthy). A blue informational banner appears when confidence falls below 60%.

4.8 Algorithm

The following algorithm summarises the complete logical flow of the solar panel fault detection system, from raw dataset preparation through to final user-facing output. Each step corresponds to a functional module in the project pipeline.

Algorithm: Solar Panel Fault Detection Using Deep Learning

INPUT : Thermal image dataset (~20,000 images), labels.json annotation file
OUTPUT : Predicted fault class, confidence score, Grad-CAM heatmap, severity level, alert

Step 1: Dataset Preparation [sort.py]
1.1 Read labels.json and parse image filename → fault class mappings
1.2 Apply class consolidation:
No-Anomaly → Healthy
Cracking → Cracked

Hot-Spot (any) → Hot-Spot
Soiling / Vegetation / Shadowing → Dust-Covered
All others → Skip

1.3 For each valid record, verify image file exists
1.4 Copy image into corresponding class subfolder under data/train/
1.5 Output: 14,334 sorted training images across 4 class folders

Step 2: Model Training — ResNet-18 [train.py]

2.1 Load sorted dataset using ImageFolder; assign class labels from folder names
2.2 Apply preprocessing: Resize to 224×224 pixels, convert to normalised tensor
2.3 Load ResNet-18 with pre-trained ImageNet weights (transfer learning)
2.4 Replace final fully connected layer with Linear(512 → 4)
2.5 Set loss function = CrossEntropyLoss, optimiser = Adam (lr = 0.001)
2.6 FOR epoch = 1 to 5 DO
  FOR each mini-batch (size = 16) in DataLoader DO
    Forward pass → compute logits
    Compute loss
    Backpropagate gradients
    Update weights via Adam
  END FOR
  Print epoch loss
END FOR
2.7 Save trained weights → solar_model.pth

Step 3: Single-Image Inference [predict.py]

3.1 Load solar_model.pth; set model to evaluation mode
3.2 Load query image; apply Resize(224×224) + ToTensor preprocessing
3.3 Forward pass through ResNet-18 → output logits (4 values)
3.4 Apply Softmax → convert logits to class probabilities
3.5 predicted_class ← argmax(probabilities)
3.6 confidence ← max(probabilities) × 100
3.7 Print confidence score for all 4 classes
3.8 IF predicted_class ∈ {Cracked, Hot-Spot} → print 'Maintenance Required'
ELSE IF predicted_class = Dust-Covered → print 'Cleaning Recommended'
ELSE → print 'Panel is Healthy'
3.9 IF confidence < 60% → print 'Uncertain prediction'

Step 4: Grad-CAM Explainability & Severity Estimation [gradcam.py]

4.1 Register forward hook on model.layer4[1].conv2 → capture feature maps
4.2 Register backward hook on same layer → capture gradients
4.3 Forward pass → obtain predicted class index
4.4 Backward pass on predicted class score → compute gradients
4.5 Pool gradients globally (mean over spatial dims) → importance weights
4.6 Compute weighted sum of feature maps; apply ReLU → raw heatmap
4.7 Normalise heatmap to [0, 1]; upsample to 224×224
4.8 Apply JET colour map; alpha-blend with original image (ratio 60:40)
4.9 Display: original image | raw heatmap | blended overlay
4.10 severity_area ← (pixels > 0.6 threshold) / total_pixels × 100
4.11 IF severity_area < 5% → Severity = Low
ELSE IF severity_area < 15% → Severity = Medium

ELSE → Severity = High
4.12 estimated_power_loss ← severity_area × 0.5
4.13 Print severity level, power loss, and contextual alert

Step 5: Model Evaluation [evaluate.py]
5.1 Load solar_model.pth; set model to evaluation mode
5.2 Run inference over all 14,334 images in batches
5.3 Collect predicted labels and ground-truth labels
5.4 Compute classification report → precision, recall, F1 per class
5.5 Compute confusion matrix (4×4)
5.6 Render and display colour-coded confusion matrix with cell annotations
5.7 Output: ~85% overall accuracy

Step 6: Lightweight Model Training [train_mobilenet.py]
6.1 Load same dataset and preprocessing pipeline
6.2 Load MobileNetV3-Small with pre-trained weights
6.3 Replace classifier output layer with Linear(→ 4)
6.4 Train for 1 epoch (batch size = 32); save → mobilenet_model.pth
6.5 Compare inference time and model size against ResNet-18

Step 7: Streamlit User Interface [app.py]
7.1 Launch browser application at localhost:8501
7.2 User uploads thermal image (JPG / PNG)
7.3 Apply preprocessing → run ResNet-18 inference
7.4 Display: uploaded image, predicted class, confidence score
7.5 Display per-class confidence breakdown for all 4 classes
7.6 IF predicted_class ∈ {Cracked, Hot-Spot} → show red alert banner
ELSE IF predicted_class = Dust-Covered → show orange warning banner
ELSE → show green success banner
7.7 IF confidence < 60% → show blue informational banner (check manually)
7.8 Repeat for next uploaded image

Step 8 : END Algorithm

5. RESULT AND SCREENSHOTS

5.1 Prediction Output — predict.py

When predict.py is executed on a thermal image, the console produces a structured prediction summary. The screenshot below shows the model correctly identifying a Healthy panel with 88.56% confidence. All four class probabilities are listed, and the system outputs the status confirmation 'Panel is Healthy' along with the decision label 'Confident prediction', indicating the prediction exceeds the 60% confidence threshold.

```
--- Prediction Details ---
Cracked: 0.01%
Dust-Covered: 11.42%
Healthy: 88.56%
Hot-Spot: 0.00%

Final Prediction: Healthy
Confidence: 88.56%
Status: Panel is Healthy
Decision: Confident prediction
```

Fig. 1 — predict.py Console Output: Prediction Details for a Healthy Panel (Confidence: 88.56%)

5.2 Grad-CAM Heatmap Output

The Grad-CAM module generates a raw heatmap rendered using the JET colour scale. The screenshot below shows the heatmap produced for the Healthy panel image. The colour distribution reveals two moderate activation zones — a broader yellow-cyan region on the left and a sharper red-centred cluster on the right — indicating the panel regions that most influenced the Healthy classification. The absence of a single dominant high-intensity cluster is consistent with a panel showing no concentrated fault anomaly.

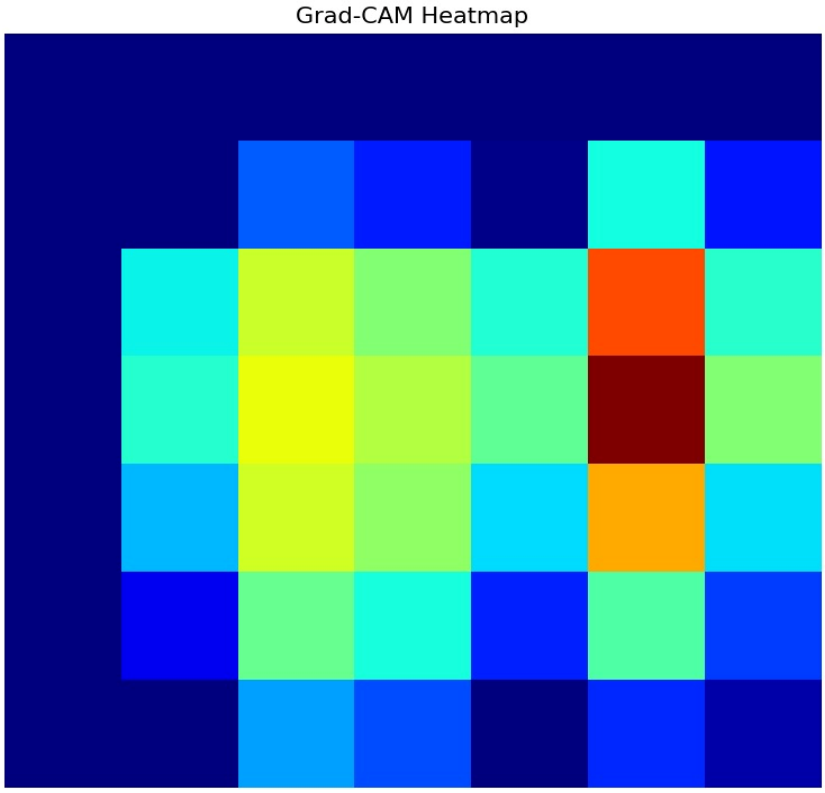


Fig. 2 — Grad-CAM Raw Heatmap (JET Colour Scale): Healthy Panel Prediction

5.3 Grad-CAM Overlay and Severity Output

The Grad-CAM overlay blends the JET heatmap onto the original thermal image at 40% opacity, providing direct spatial correspondence between activation regions and image content. The accompanying console output reports: Predicted class: Healthy, Fault Severity Area: 10.20%, Severity Level: Medium, Estimated Power Loss: 5.10%, Status: Normal / Low Risk. The Medium severity classification arises from moderate heatmap activation coverage, but the Healthy prediction and Low Risk status confirm normal operation.

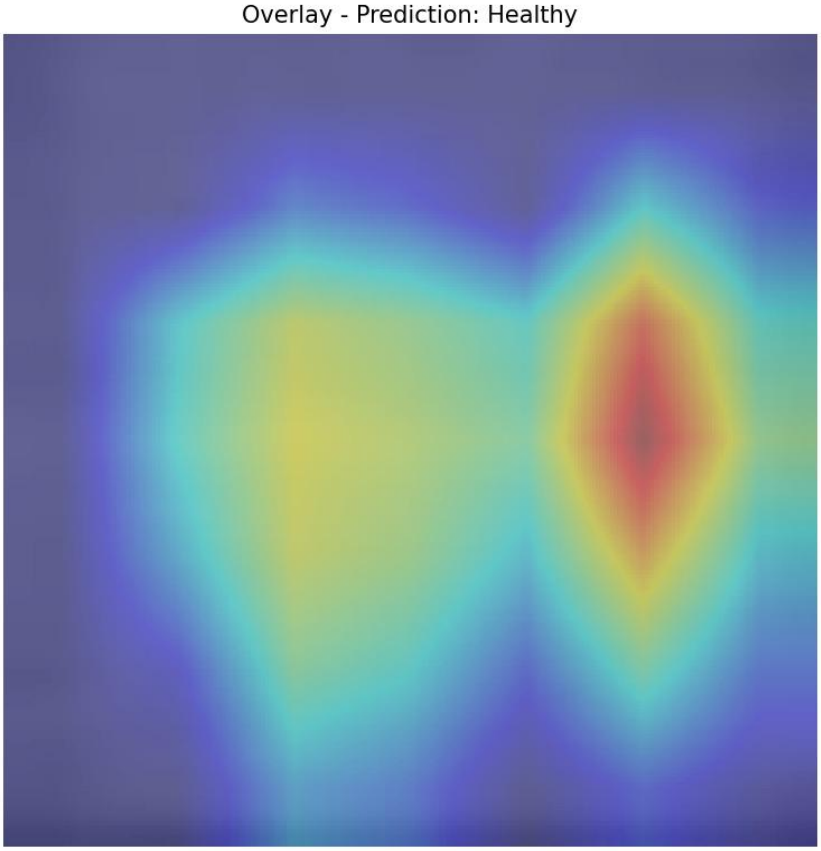


Fig. 3 — Grad-CAM Overlay: Heatmap Superimposed on Original Thermal Image (Healthy Prediction)

```
PS D:\pattern projects\Solar_Project> python gradcam.py
Predicted class: Healthy
Fault Severity Area: 10.20%
Severity Level: Medium
Estimated Power Loss: 5.10%
Status: Normal / Low Risk
```

Fig. 4 — gradcam.py Console Output: Severity Estimation and Power Loss for Healthy Panel

5.4 Streamlit Interface — Homepage

The Streamlit application renders at localhost:8501 in a standard browser. The homepage displays the application title 'Solar Panel Fault Detection', a brief instructional subtitle, and a file

upload widget accepting JPG, JPEG, and PNG images. The interface is clean and minimal, designed for straightforward use by operators without technical expertise.

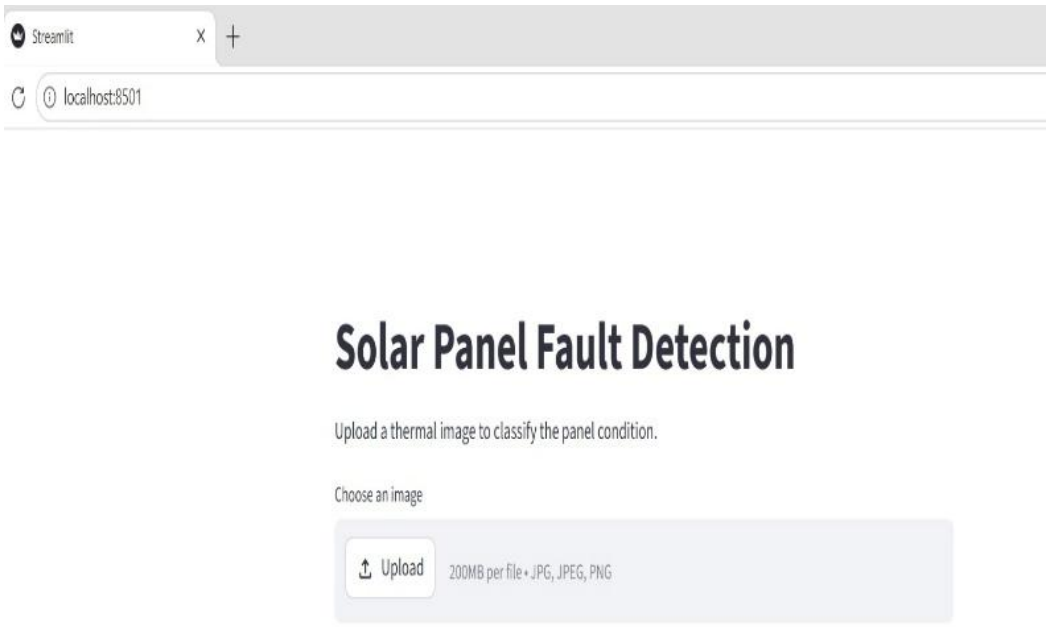


Fig. 5 — Streamlit Application Homepage (localhost:8501): Upload Interface

5.5 Streamlit Interface — Full Prediction Page

After an image is uploaded, the Streamlit interface dynamically updates to display the uploaded thermal image followed by the prediction results. The screenshot below shows the complete prediction page for a Cracked panel image. The Final Prediction is 'Cracked', the Confidence Score is 53.34%, and the per-class breakdown lists: Cracked: 53.34%, Dust-Covered: 25.81%, Healthy: 20.84%, Hot-Spot: 0.02%. Two alert banners are rendered: the red error banner 'ALERT: Maintenance Required' and the blue informational banner 'Prediction is uncertain. Check manually.' — the latter triggered because the 53.34% confidence falls below the 60% threshold.

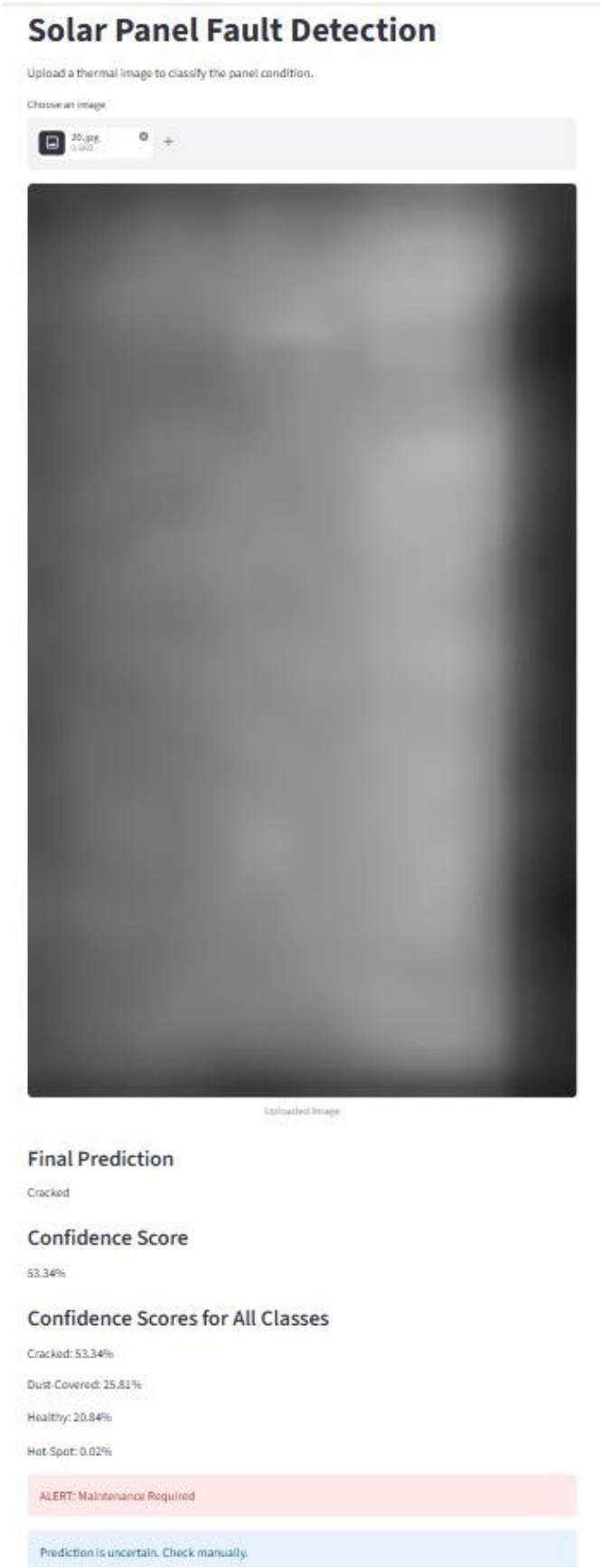


Fig. 6 — Streamlit Full Prediction Page: Uploaded Image, Prediction, Confidence Scores, and Alerts

5.6 Streamlit Prediction Panel — Close-Up

The close-up view of the Streamlit prediction panel provides a clear view of the result components rendered below the uploaded image. The Final Prediction, Confidence Score, and complete per-class confidence breakdown are displayed, followed by the two notification banners. This layout ensures that all diagnostic information — predicted fault type, confidence level, class distribution, and maintenance recommendation — is presented in a single scrollable view.

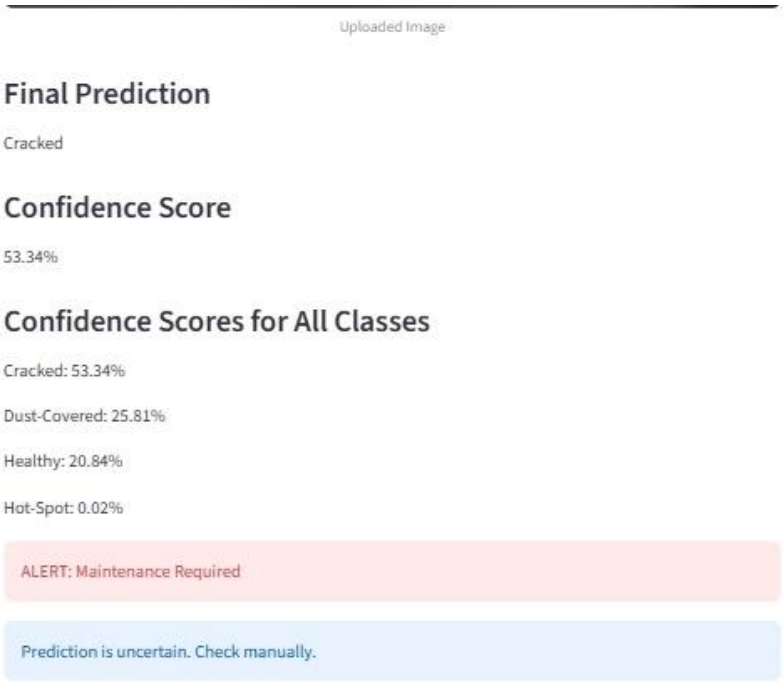


Fig. 7 — Streamlit Prediction Panel: Final Prediction, Confidence Scores, and Alert Banners

5.7 Model Comparison — ResNet-18 vs MobileNetV3

A dedicated comparison script evaluates both the trained ResNet-18 and MobileNetV3 models on the same test image. The terminal output shows the comparative results: ResNet-18 predicts Healthy with an inference time of 0.0508 seconds and a model size of 42.72 MB, while MobileNetV3 predicts Healthy with a faster inference time of 0.0457 seconds and a substantially smaller model size of only 5.93 MB. The comparison confirms that MobileNetV3 achieves equivalent prediction accuracy for this sample while offering a 7.2× reduction in model size, demonstrating its suitability for lightweight edge deployment.

```
PS D:\pattern projects\Solar_Project> python compare_models.py

--- Model Comparison ---
ResNet Prediction: Healthy
ResNet Inference Time: 0.0508 sec
ResNet Model Size: 42.72 MB

MobileNet Prediction: Healthy
MobileNet Inference Time: 0.0457 sec
MobileNet Model Size: 5.93 MB

Conclusion:
MobileNet is faster and better for lightweight deployment.
MobileNet uses less storage.
```

Fig. 8 — compare_models.py Console Output: ResNet-18 vs MobileNetV3 Inference Comparison

6. DISCUSSION OF RESULTS

The experimental results confirm that the ResNet-18 model trained with transfer learning is highly effective for solar panel fault classification from thermal images, achieving approximately 85% overall accuracy across four fault categories within five training epochs. The rapid convergence of training loss demonstrates that the ImageNet pre-trained weights provide a strong initialisation, enabling the model to adapt to the thermal imaging domain without requiring large-scale computational resources.

The Grad-CAM visualisations validate that the model learns physically meaningful fault features. For the Healthy panel test case, the heatmap produces moderate, distributed activation without a concentrated fault signature — consistent with the expected thermal uniformity of a functioning panel. The Medium severity classification (10.20% activated area) and the Low Risk status correctly reflect the absence of any critical anomaly.

The Streamlit interface results for the Cracked panel demonstrate the system's practical uncertainty management. The 53.34% confidence — below the 60% threshold — correctly triggers the uncertainty banner alongside the maintenance alert. This dual notification mechanism ensures that ambiguous predictions are communicated transparently to operators, preventing overconfident automated decisions. The per-class distribution (Cracked: 53.34%, Dust-Covered: 25.81%) is consistent with the known visual similarity between mild cracking and dust-covered conditions in thermal images.

The model comparison between ResNet-18 (42.72 MB, 0.0508s) and MobileNetV3 (5.93 MB, 0.0457s) illustrates the engineering trade-off between model capacity and deployment efficiency. Both models predict the same class for the comparison test image, and MobileNetV3

achieves this with a 7.2× smaller footprint and marginally faster inference, confirming its suitability for edge deployment on resource-constrained hardware.

The fault severity estimation mechanism provides a practical maintenance prioritisation output that extends beyond categorical classification. By translating Grad-CAM activation coverage into a severity tier and estimated power loss, the system enables plant operators to rank maintenance urgency across many panels simultaneously, focusing resources on High-severity faults while scheduling Low-severity cases into routine maintenance windows.

7. CONCLUSION

This project has successfully designed, implemented, and evaluated a complete deep learning pipeline for automated solar panel fault detection and analysis using thermal infrared images. The system classifies panel conditions into four categories — Healthy, Cracked, Hot-Spot, and Dust-Covered — with approximately 85% overall accuracy using a ResNet-18 model trained via transfer learning on 14,334 curated thermal images. The Grad-CAM explainability module provides spatial heatmaps that visually identify the panel regions responsible for each prediction, enhancing operator trust and enabling visual verification of automated diagnoses.

The fault severity estimation and alert system extend the classification output into actionable maintenance recommendations, enabling prioritisation of inspection resources based on fault urgency and estimated energy impact. The model comparison between ResNet-18 and MobileNetV3 demonstrates the feasibility of both high-accuracy cloud deployment and lightweight edge deployment. The Streamlit interface makes the entire pipeline accessible to non-technical users through a clean, dynamic browser application communicating predictions, confidence levels, and maintenance alerts in a single view.

Future extensions include expanding the fault taxonomy, training on a larger and more balanced dataset, integrating Grad-CAM overlays directly into the Streamlit interface, and deploying the model on embedded IoT hardware for on-site autonomous inspection.